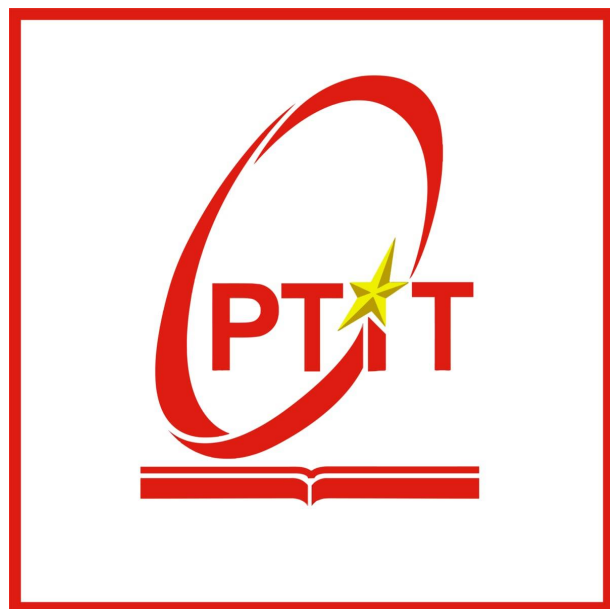


**HỌC VIỆN CÔNG NGHỆ BƯU CHÍNH VIỄN THÔNG
KHOA CÔNG NGHỆ THÔNG TIN**

HỆ QUẢN TRỊ CƠ SỞ DỮ LIỆU



BÁO CÁO BÀI TẬP LỚN- NHÓM 6

**HỆ THỐNG QUẢN LÝ GIẢI ĐUA XE CÔNG THỨC
F1**

Giảng viên hướng dẫn: PGS.TS. Nguyễn Trọng Khánh

Nhóm sinh viên thực hiện: Nguyễn Văn Hiếu - B23DCCE033

Phạm Nhật Minh - B23DCCE066

Lê Minh Đức - B23DCVT095

Hà Nội, Tháng 4/2026

Mục lục

1	CHƯƠNG 1: GIỚI THIỆU	7
1.1	Tổng quan về DBMS	7
1.2	Vai trò của DBMS trong hệ thống thông tin	7
1.3	Giới thiệu bài toán — F1 Championship Management System	7
1.4	Mục tiêu báo cáo	8
2	CHƯƠNG 2: CƠ SỞ LÝ THUYẾT	9
2.1	Khái niệm DBMS	9
2.2	Các mô hình dữ liệu	9
2.3	Kiến trúc 3 mức (Three-Level Architecture)	9
2.4	Các khái niệm cơ bản	10
3	CHƯƠNG 3: MÔ TẢ BÀI TOÁN & PHÂN TÍCH YÊU CẦU	11
3.1	Mô tả hệ thống	11
3.2	Người dùng hệ thống	11
3.3	Yêu cầu chức năng (Functional Requirements)	12
3.4	Yêu cầu phi chức năng (Non-Functional Requirements)	12
4	CHƯƠNG 4: LOGIC NGHIỆP VỤ	13
4.1	Quy trình nghiệp vụ tổng thể	13
4.2	Business Rules (Quy tắc nghiệp vụ)	13
4.3	Trigger — Ràng buộc nghiệp vụ tại tầng Database	14
4.3.1	Trigger 1: trg_limit_2_riders	14
4.3.2	Trigger 2 & 3: trg_check_time_insert / trg_check_time_update	15
4.4	Transaction — Đảm bảo tính nguyên tử	16
4.5	Stored Procedure — sp_calculate_points	16
4.6	Views — Bảng ảo tổng hợp BXH	18
5	CHƯƠNG 5: THIẾT KẾ CƠ SỞ DỮ LIỆU	20
5.1	Mô hình thực thể — liên kết (ERD)	20
5.2	Mô tả chi tiết các thực thể	20
5.3	Mô hình quan hệ	21

5.4	Chuẩn hóa (Normalization)	21
5.4.1	1NF — First Normal Form	21
5.4.2	2NF — Second Normal Form	21
5.4.3	3NF — Third Normal Form	21
5.4.4	BCNF — Boyce-Codd Normal Form	22
6	CHƯƠNG 6: TRIỂN KHAI	23
6.1	Môi trường và công nghệ	23
6.2	Schema — Tạo cấu trúc bảng (DDL)	23
6.3	Nạp dữ liệu mẫu (DML — INSERT)	25
6.4	Các câu truy vấn SQL quan trọng	31
6.5	Index — Tối ưu hóa hiệu năng	33
7	CHƯƠNG 7: ĐÁNH GIÁ	34
7.1	Đánh giá thiết kế cơ sở dữ liệu	34
7.1.1	Điểm mạnh	34
7.2	Hạn chế hiện tại	34
7.3	Khả năng mở rộng	35
8	CHƯƠNG 8: KẾT LUẬN	36
8.1	Tóm tắt kết quả	36
8.2	Bài học rút ra	36
8.3	Hướng phát triển	37
	Tài Liệu Tham Khảo	38

Danh sách hình vẽ

4.1	Vòng đời dữ liệu của một chặng đua trong hệ thống	13
4.2	Luồng xử lý của Trigger trg_limit_2_riders	14
4.3	Luồng Transaction của Module 2 — Nhập kết quả	16
5.1	Sơ đồ ERD của hệ thống F1 Championship Management	20

Danh sách bảng

3.1	Các module của hệ thống	11
4.1	Hệ thống tính điểm F1	14
5.1	Bảng CHAMPIONSHIPS — Mùa giải	20
5.2	Bảng CONTRACTS — Hợp đồng tay đua — đội	20
5.3	Bảng RESULTS — Kết quả thi đấu	21
6.1	Bảng công nghệ sử dụng	23
6.2	Chiến lược Index của hệ thống	33
7.1	Tổng hợp các đối tượng DBMS trong hệ thống	34

Listings

4.1	Trigger trg_limit_2_riders — Giới hạn 2 tay đua/đội/chặng	14
4.2	Trigger kiểm tra tính hợp lệ của thời gian kết thúc	15
4.3	Stored Procedure sp_calculate_points — Tính điểm F1 tự động	16
4.4	View v_race_performance — Nguồn dữ liệu gốc cho BXH	18
4.5	View v_driver_standings — BXH tay đua toàn mùa giải	18
4.6	View v_team_standings — BXH đội đua toàn mùa giải	18
6.1	DDL — Tạo toàn bộ cấu trúc bảng	23
6.2	INSERT — Dữ liệu mẫu mùa giải F1 2025 và F1 2026	25
6.3	SELECT với JOIN — Lấy danh sách đăng ký một chặng	31
6.4	SELECT với GROUP BY + HAVING — Đội có điểm trên 50	32
6.5	INSERT với ON DUPLICATE KEY UPDATE — Upsert kết quả	32
6.6	SELECT với Subquery — BXH đến một chặng cụ thể	32
6.7	Tạo 4 index bổ sung tối ưu hiệu năng	33

Bảng Giải Thích Thuật Ngữ

Thuật ngữ	Giải thích
ACID	Tập hợp 4 thuộc tính (Atomicity, Consistency, Isolation, Durability) đảm bảo tính tin cậy của giao dịch cơ sở dữ liệu.
MVCC	(Multi-Version Concurrency Control) Cơ chế hỗ trợ truy cập đồng thời bằng cách cho phép nhiều phiên bản dữ liệu tồn tại cùng lúc.
DBA	(Database Administrator) Người quản trị chịu trách nhiệm vận hành, bảo mật và tối ưu hóa hệ thống cơ sở dữ liệu.
InnoDB engine	Bộ máy lưu trữ mặc định của MySQL hỗ trợ giao dịch ACID, khóa mức dòng và tính năng phục hồi sau sự cố.
B+ Tree	Cấu trúc dữ liệu cây tự cân bằng giúp tối ưu việc tìm kiếm và truy xuất dữ liệu theo khối trong DBMS.
Buffer pool	Vùng nhớ RAM dùng để lưu đệm dữ liệu và chỉ mục, giúp giảm thiểu việc đọc/ghi trực tiếp từ ổ cứng.
Redo log	Nhật ký lưu lại các thay đổi dữ liệu để phục hồi trạng thái hệ thống khi xảy ra sự cố đột ngột.
Undo log	Nhật ký lưu lại trạng thái cũ của dữ liệu để phục vụ việc hoàn tác (Rollback) và cơ chế MVCC.
Trigger	Đối tượng mã SQL tự động thực thi khi có sự kiện INSERT, UPDATE hoặc DELETE trên bảng.
Transaction	Một đơn vị công việc logic gồm một chuỗi các thao tác SQL, đảm bảo thực hiện hoàn toàn hoặc không thực hiện gì.
Stored Procedure	Tập hợp các câu lệnh SQL được biên dịch và lưu trữ sẵn trên Server để thực thi các nghiệp vụ lặp lại.
Views	Bảng ảo giúp trình bày dữ liệu từ một hoặc nhiều bảng thông qua một câu lệnh truy vấn có sẵn.
Index	Cấu trúc dữ liệu giúp tăng tốc độ tìm kiếm bản ghi, tương tự như mục lục của một cuốn sách.
JWT Authentication	(JSON Web Token) Phương thức xác thực an toàn giữa các thành phần của hệ thống thông qua chuỗi mã hóa.
CTE	(Common Table Expression) Tập kết quả tạm thời giúp câu lệnh SQL phức tạp trở nên rõ ràng và dễ quản lý hơn.

CHƯƠNG 1: GIỚI THIỆU

1.1 Tổng quan về DBMS

Hệ Quản Trị Cơ Sở Dữ Liệu (*Database Management System — DBMS*) là phần mềm trung gian đứng giữa người dùng/ứng dụng và dữ liệu được lưu trữ vật lý, cung cấp các cơ chế để định nghĩa, tạo lập, duy trì và kiểm soát truy cập đến cơ sở dữ liệu. Thay vì mỗi ứng dụng tự quản lý file riêng lẻ — dẫn đến trùng lặp dữ liệu, không nhất quán và khó bảo trì — DBMS cung cấp một kho dữ liệu tập trung, nhất quán và có thể truy cập đồng thời bởi nhiều người dùng.

Các DBMS hiện đại như MySQL, PostgreSQL, Oracle, SQL Server đều xây dựng trên nền tảng mô hình quan hệ (*Relational Model*) được Edgar F. Codd đề xuất năm 1970. Trong mô hình này, dữ liệu được tổ chức thành các bảng (*table*) gồm nhiều hàng (*row*) và cột (*column*), các bảng liên kết với nhau qua khóa ngoại (*foreign key*), và ngôn ngữ SQL (*Structured Query Language*) được dùng để thao tác dữ liệu một cách khai báo (*declarative*).

1.2 Vai trò của DBMS trong hệ thống thông tin

Trong kiến trúc hệ thống thông tin hiện đại, DBMS đóng vai trò là lớp nền tảng không thể thiếu:

- **Tính toàn vẹn dữ liệu (*Data Integrity*):** DBMS tự động kiểm tra và đảm bảo dữ liệu luôn hợp lệ thông qua các ràng buộc (*constraints*), trigger và transaction.
- **Tính nhất quán (*Consistency*):** Nhiều người dùng truy cập đồng thời nhưng mỗi người đều nhìn thấy trạng thái nhất quán của dữ liệu, nhờ cơ chế ACID transaction và MVCC (*Multi-Version Concurrency Control*).
- **Tính bảo mật (*Security*):** DBMS cung cấp hệ thống phân quyền chi tiết — mỗi user chỉ truy cập được những gì được cho phép.
- **Tính hiệu năng (*Performance*):** Index, query optimizer và caching giúp truy xuất dữ liệu nhanh ngay cả khi bảng có hàng triệu dòng.
- **Tính bền vững (*Durability*):** Dữ liệu được đảm bảo không bị mất sau khi commit, kể cả khi server gặp sự cố đột ngột.

1.3 Giới thiệu bài toán — F1 Championship Management System

Formula 1 (F1) là giải đua xe thể thức một hàng đầu thế giới, với hàng chục đội đua, hơn 20 tay đua chuyên nghiệp và lịch thi đấu kéo dài gần một năm. Mỗi mùa giải gồm nhiều chặng đua (*Grand Prix*) tại các quốc gia khác nhau.

F1 Championship Management System là hệ thống phần mềm quản lý toàn bộ vòng đời dữ liệu của nhiều mùa giải F1:

- Quản lý thông tin đội đua, tay đua, hợp đồng ký kết
- Quản lý lịch thi đấu và các chặng đua
- Cho phép đăng ký tay đua vào từng chặng đua
- Ghi nhận kết quả thi đấu sau mỗi chặng
- Tự động tính điểm theo hệ thống tính điểm F1 quốc tế
- Cung cấp bảng xếp hạng (BXH) tay đua và đội đua theo thời gian thực

Hệ thống là ứng dụng web full-stack gồm: Frontend React.js (giao diện người dùng), Backend Node.js/Express (REST API), và MySQL/MariaDB (cơ sở dữ liệu).

1.4 Mục tiêu báo cáo

Báo cáo này nhằm:

1. Trình bày quá trình phân tích, thiết kế và triển khai cơ sở dữ liệu cho hệ thống F1 Championship Management.
2. Minh họa cách áp dụng các khái niệm DBMS học thuật (chuẩn hóa, ràng buộc toàn vẹn, transaction ACID, index, view, stored procedure, trigger) vào bài toán thực tế.
3. Phân tích chi tiết nghiệp vụ logic và cách cơ sở dữ liệu hỗ trợ các quy trình nghiệp vụ đó.
4. Đánh giá các quyết định thiết kế và gợi ý hướng cải thiện.

CHƯƠNG 2: CƠ SỞ LÝ THUYẾT

2.1 Khái niệm DBMS

Định nghĩa: DBMS là tập hợp các chương trình cho phép người dùng tạo lập, duy trì và truy xuất cơ sở dữ liệu. DBMS cung cấp giao diện giữa người dùng và cơ sở dữ liệu, đồng thời quản lý tất cả truy cập vào dữ liệu.

Các thành phần ngôn ngữ chính của DBMS:

- **DDL (*Data Definition Language*):** Ngôn ngữ định nghĩa cấu trúc — `CREATE TABLE`, `ALTER TABLE`, `DROP TABLE`
- **DML (*Data Manipulation Language*):** Ngôn ngữ thao tác dữ liệu — `SELECT`, `INSERT`, `UPDATE`, `DELETE`.
- **DCL (*Data Control Language*):** Ngôn ngữ kiểm soát truy cập — `GRANT`, `REVOKE`.
- **TCL (*Transaction Control Language*):** Ngôn ngữ kiểm soát giao dịch — `BEGIN`, `COMMIT`, `ROLLBACK`.

2.2 Các mô hình dữ liệu

Mô hình quan hệ (*Relational Model*) — được sử dụng trong dự án này:

Dữ liệu được tổ chức thành các quan hệ (*relation*), biểu diễn dưới dạng bảng 2 chiều. Mỗi hàng (*tuple*) đại diện cho một thực thể, mỗi cột (*attribute*) đại diện cho một thuộc tính. Mỗi quan hệ giữa các bảng được thiết lập qua Khóa ngoại (*Foreign Key*).

Mô hình phi quan hệ (*NoSQL*):

Gồm nhiều dạng: Document Store (MongoDB), Key-Value Store (Redis), Column Family (Cassandra), Graph Database (Neo4j). Phù hợp cho dữ liệu phi cấu trúc hoặc yêu cầu mở rộng ngang cực lớn.

Lý do chọn mô hình quan hệ cho bài toán F1:

Dữ liệu F1 có cấu trúc rõ ràng, nhiều mối quan hệ phức tạp giữa các thực thể (tay đua — đội — hợp đồng — chặng — kết quả), và yêu cầu ràng buộc toàn vẹn chặt chẽ. Mô hình quan hệ với MySQL/InnoDB đáp ứng đầy đủ các yêu cầu này.

2.3 Kiến trúc 3 mức (Three-Level Architecture)

Kiến trúc ANSI/SPARC định nghĩa 3 mức nhìn vào cơ sở dữ liệu:

- **Mức Ngoài:** Mỗi nhóm người dùng nhìn thấy một “view” khác nhau của dữ liệu. Trong dự án F1, trang BXH chỉ thấy điểm và xếp hạng qua `v_driver_standings`.

- **Mức Khái niệm:** Toàn bộ schema — 7 bảng với các mối quan hệ, ràng buộc, trigger, stored procedure. Đây là tầng DBA làm việc.
- **Mức Vật lý:** InnoDB engine tổ chức dữ liệu thực sự trên đĩa theo B+ Tree, quản lý buffer pool, redo log, undo log.

2.4 Các khái niệm cơ bản

Khóa chính (Primary Key — PK): Cột hoặc tập cột xác định duy nhất mỗi hàng trong bảng. Ví dụ: `driver_code` trong bảng `DRIVERS`.

Khóa ngoại (Foreign Key — FK): Cột tham chiếu đến PK của bảng khác. Ví dụ: `RESULTS.entry_id` tham chiếu đến `RACE_ENTRIES.entry_id`.

Khóa dự tuyển (Candidate Key): Bất kỳ tập thuộc tính nào có thể xác định duy nhất một hàng. Trong `RACE_ENTRIES`, cặp (`race_code`, `contract_id`) là candidate key.

Ràng buộc toàn vẹn (Integrity Constraint):

- **Domain Constraint:** Giới hạn miền giá trị (NOT NULL, CHECK, ENUM)
- **Entity Integrity:** PK không được NULL
- **Referential Integrity:** FK phải tham chiếu đến giá trị PK tồn tại
- **Semantic Integrity:** Ràng buộc nghiệp vụ tùy chỉnh (Trigger)

CHƯƠNG 3: MÔ TẢ BÀI TOÀN & PHÂN TÍCH YÊU CẦU

3.1 Mô tả hệ thống

Hệ thống gồm 4 module chính:

Bảng 3.1: Các module của hệ thống

Module	Tên	Vai trò
Module 1	Register Racing	Đăng ký tay đua tham gia chặng đua
Module 2	Update Results	Nhập kết quả thi đấu sau chặng
Module 3	Driver Standings	Xem bảng xếp hạng tay đua
Module 4	Team Standings	Xem bảng xếp hạng đội đua

3.2 Người dùng hệ thống

Ban tổ chức / Staff nhập liệu:

- Đăng ký tay đua vào từng chặng đua (Module 1)
- Nhập kết quả sau khi chặng kết thúc (Module 2)
- Xem bảng xếp hạng để kiểm tra và công bố kết quả (Module 3, 4)

Quản trị viên hệ thống (DBA):

- Khởi tạo và bảo trì cơ sở dữ liệu
- Thêm mùa giải, chặng đua, đội đua, tay đua mới
- Theo dõi hiệu năng và backup định kỳ

Người xem / Khán giả (Read-only): Xem bảng xếp hạng, không có quyền ghi dữ liệu.

NOTE

Do phạm vi của môn học Database Management Systems tập trung vào thiết kế và triển khai cơ sở dữ liệu, nhóm ưu tiên trình bày và minh họa các chức năng chính của hệ thống. Vì vậy, phần demo sẽ được thực hiện theo hướng tổng thể, không đi sâu vào việc phân quyền chi tiết giữa các vai trò (như người dùng, nhân viên, v.v.) mà thay vào đó thể hiện chung các chức năng của hệ thống.

3.3 Yêu cầu chức năng (Functional Requirements)

- FR-01** Hệ thống quản lý thông tin các mùa giải F1 (thêm, sửa, xem).
- FR-02** Hệ thống quản lý danh sách đội đua và tay đua, bao gồm thông tin cá nhân và lịch sử hợp đồng.
- FR-03** Hệ thống cho phép đăng ký tay đua vào từng chặng đua, với ràng buộc mỗi đội tối đa 2 tay đua/chặng.
- FR-04** Hệ thống cho phép nhập kết quả thi đấu (thời gian kết thúc, số vòng hoàn thành, trạng thái) cho từng tay đua.
- FR-05** Hệ thống tự động tính điểm theo hệ thống tính điểm F1 quốc tế (P1=25, P2=18, ..., P10=1) ngay sau khi kết quả được lưu.
- FR-06** Hệ thống cung cấp bảng xếp hạng tay đua và đội đua theo thời gian thực, hỗ trợ lọc theo từng chặng cụ thể.
- FR-07** Hệ thống cho phép xem lịch sử kết quả chi tiết của từng tay đua và từng đội qua các chặng.
- FR-08** Hệ thống hỗ trợ sửa lại kết quả đã nhập (điều chỉnh sau khi xem xét lại).

3.4 Yêu cầu phi chức năng (Non-Functional Requirements)

Hiệu năng

Bảng xếp hạng tải trong < 2 giây; Module nhập kết quả phản hồi trong < 3 giây kể cả bước tính điểm.

Toàn vẹn dữ liệu

Dữ liệu nhất quán tại mọi thời điểm. Không thể nhập kết quả vi phạm nghiệp vụ.

Bảo mật

Thông tin đăng nhập DB không hardcode. Mọi truy vấn dùng Prepared Statements chống SQL Injection.

Khả dụng

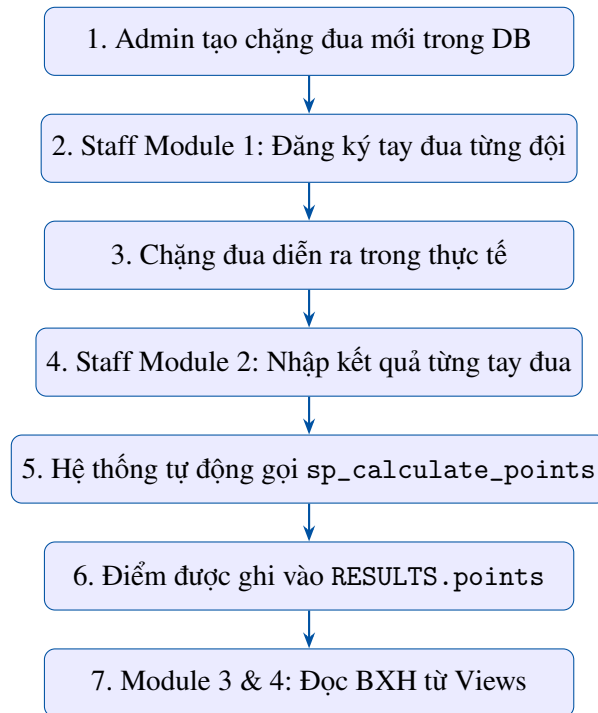
Hỗ trợ nhiều người dùng đồng thời không block nhau (InnoDB MVCC).

Phục hồi

Chiến lược backup có thể khôi phục dữ liệu đến thời điểm bất kỳ.

CHƯƠNG 4: LOGIC NGHIỆP VỤ

4.1 Quy trình nghiệp vụ tổng thể



Hình 4.1: Vòng đời dữ liệu của một chặng đua trong hệ thống

4.2 Business Rules (Quy tắc nghiệp vụ)

BR-01 — Giới hạn tay đua mỗi đội

Mỗi đội đua chỉ được đăng ký tối đa 2 tay đua cho mỗi chặng đua. Ràng buộc này được thực thi bằng **Trigger** tại tầng Database.

BR-02 — Hệ thống tính điểm F1

Điểm theo thứ hạng thời gian, chỉ tay đua hoàn thành (`status = 'Finished'`) mới được xếp hạng và tính điểm.

BR-03 — Tính hợp lệ thời gian

`end_time` phải lớn hơn `start_time` của chặng. Thực thi bằng Trigger bảo vệ cả INSERT và UPDATE.

BR-04 — Xếp hạng khi bằng điểm

Ưu tiên: (1) Tổng điểm giảm dần; (2) Tổng thời gian tăng dần.

BR-05 — Điểm đội

Bảng tổng điểm của TẤT CẢ tay đua của đội tại chặng đó (không lấy trung bình).

BR-06 — Lịch sử hợp đồng

Khi chuyển đội: hợp đồng cũ `is_active = 0`, INSERT hợp đồng mới với `is_active = 1`.

Bảng điểm F1 chuẩn (BR-02):

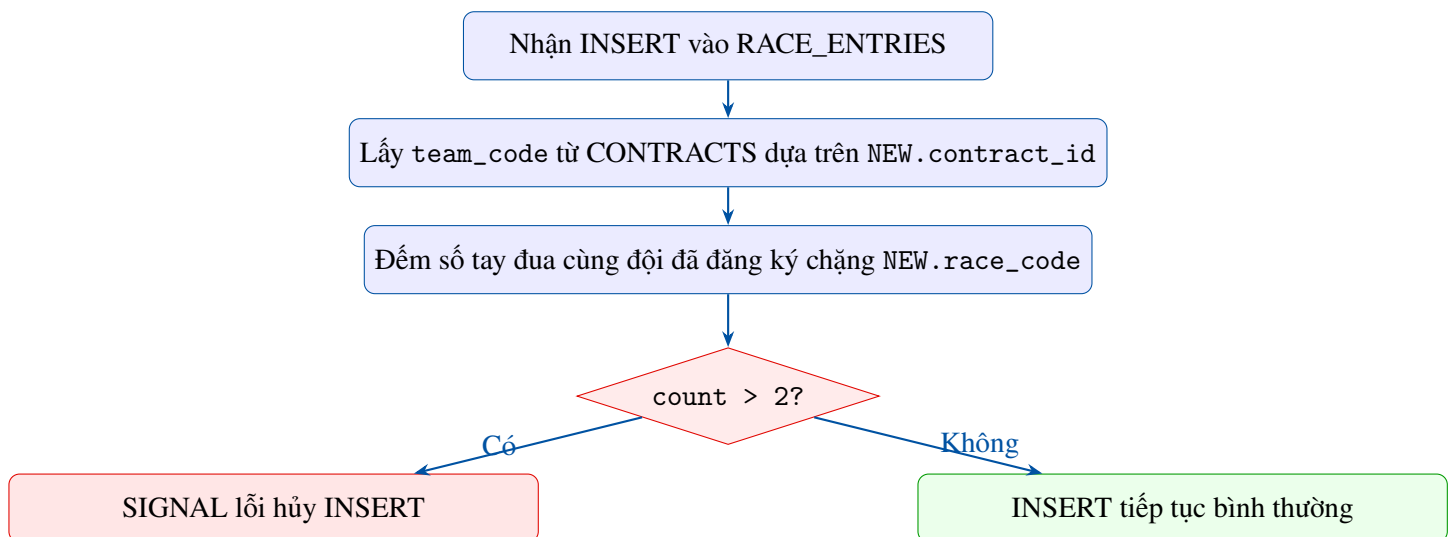
Bảng 4.1: Hệ thống tính điểm F1

Hạng	Điểm	Hạng	Điểm
P1	25	P6	8
P2	18	P7	6
P3	15	P8	4
P4	12	P9	2
P5	10	P10	1
P11 trở đi: 0		DNF/Accident: 0	

4.3 Trigger — Ràng buộc nghiệp vụ tại tầng Database**4.3.1 Trigger 1: `trg_limit_2_riders`**

Mục đích: Chặn đăng ký tay đua thứ 3 của cùng một đội vào cùng một chặng.

Thời điểm kích hoạt: BEFORE INSERT trên bảng `RACE_ENTRIES`.

Hình 4.2: Luồng xử lý của Trigger `trg_limit_2_riders`

```

1 DELIMITER //
2 CREATE TRIGGER trg_limit_2_riders
3 BEFORE INSERT ON RACE_ENTRIES

```

```

4  FOR EACH ROW
5  BEGIN
6      DECLARE v_team_id VARCHAR(10);
7      DECLARE v_count INT;
8      -- Lay team_code của tay đua sắp đăng ký
9      SELECT team_code INTO v_team_id
10     FROM CONTRACTS WHERE contract_id = NEW.contract_id;
11     -- Dem tay đua cùng đội đã đăng ký chặng này
12     SELECT COUNT(*) INTO v_count
13     FROM RACE_ENTRIES re
14     JOIN CONTRACTS c ON re.contract_id = c.contract_id
15     WHERE re.race_code = NEW.race_code
16           AND c.team_code = v_team_id;
17     -- Neu đã đủ 2 -> tu chối
18     IF v_count >= 2 THEN
19         SIGNAL SQLSTATE '45000'
20         SET MESSAGE_TEXT = 'Loi: Moi doi chi duoc toi da 2 tay đua!';
21     END IF;
22 END //
23 DELIMITER ;

```

Listing 4.1: Trigger trg_limit_2_riders — Giới hạn 2 tay đua/đội/chặng

4.3.2 Trigger 2 & 3: trg_check_time_insert / trg_check_time_update

Mục đích: Đảm bảo end_time > start_time của chặng tương ứng.

Thời điểm kích hoạt: BEFORE INSERT và BEFORE UPDATE trên bảng RESULTS.

```

1  DELIMITER //
2  CREATE TRIGGER trg_check_time_insert
3  BEFORE INSERT ON RESULTS
4  FOR EACH ROW
5  BEGIN
6      DECLARE v_start_time DATETIME(3);
7      IF NEW.status = 'Finished' AND NEW.end_time IS NOT NULL THEN
8          SELECT r.start_time INTO v_start_time
9          FROM RACE_ENTRIES re
10         JOIN RACES r ON re.race_code = r.race_code
11         WHERE re.entry_id = NEW.entry_id;
12
13         IF NEW.end_time <= v_start_time THEN
14             SIGNAL SQLSTATE '45000'

```

```

15      SET MESSAGE_TEXT =
16          'Lỗi: Thời gian kết thúc phải lớn hơn thời gian bắt đầu
           chang!';
17      END IF;
18  END IF;
19  END //
20  DELIMITER ;

```

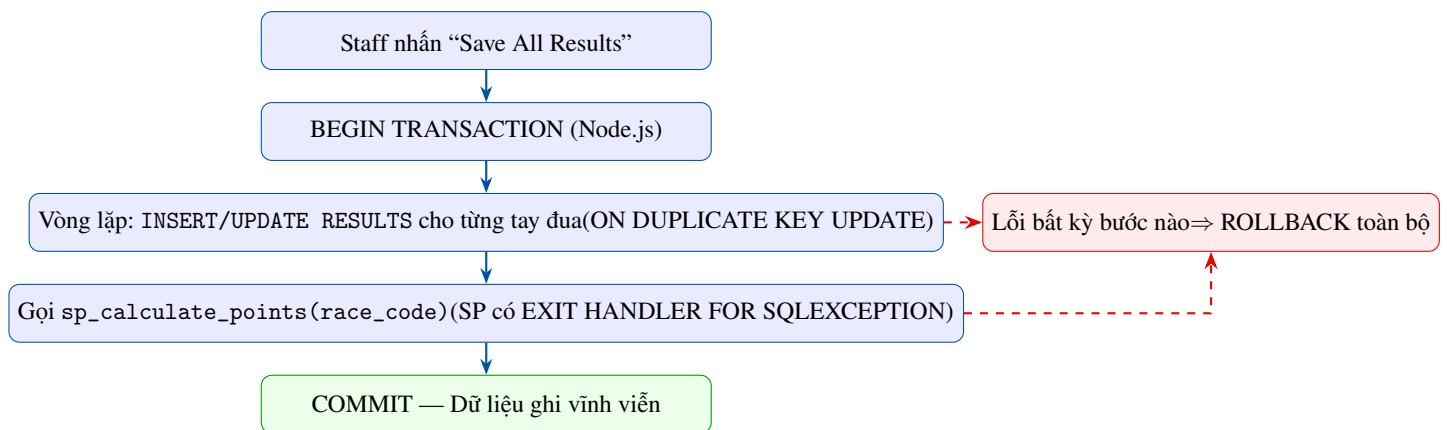
Listing 4.2: Trigger kiểm tra tính hợp lệ của thời gian kết thúc

Trigger `trg_check_time_update` có cấu trúc hoàn toàn tương tự nhưng áp dụng cho thao tác UPDATE. Cần 2 trigger riêng biệt để bao phủ cả 2 trường hợp nhập mới và chỉnh sửa lại kết quả.

Lý do đặt ràng buộc ở tầng Database thay vì chỉ ở tầng ứng dụng: Nếu chỉ kiểm tra ở Frontend hay Backend Node.js, người dùng có thể bypass bằng cách gọi API trực tiếp (Postman, curl) hoặc kết nối DB bằng MySQL Workbench. Trigger đảm bảo ràng buộc được áp dụng tại **MỌI ĐIỂM TRUY CẬP** vào Database.

4.4 Transaction — Đảm bảo tính nguyên tử

Đây là nghiệp vụ phức tạp nhất trong hệ thống, cần Transaction để đảm bảo tính ACID:



Hình 4.3: Luồng Transaction của Module 2 — Nhập kết quả

4.5 Stored Procedure — `sp_calculate_points`

Mục đích: Tính điểm theo thứ hạng thời gian và lưu vật lý vào `RESULTS.points`. Được gọi tự động sau mỗi lần lưu kết quả.

```

1  DELIMITER //
2  CREATE PROCEDURE sp_calculate_points(IN p_race_code VARCHAR(10))

```

```

3 BEGIN
4     -- Khai bao Handler: Gap loi se tu dong ROLLBACK
5     DECLARE EXIT HANDLER FOR SQLEXCEPTION
6     BEGIN
7         ROLLBACK;
8         RESIGNAL;
9     END;
10
11    START TRANSACTION;
12
13    -- Buoc 1: Reset diem chang ve 0 de tinh lai tu dau
14    UPDATE RESULTS res
15    JOIN RACE_ENTRIES re ON res.entry_id = re.entry_id
16    SET res.points = 0
17    WHERE re.race_code = p_race_code;
18
19    -- Buoc 2 & 3: Xep hang thoi gian va cap nhat diem vat ly
20    UPDATE RESULTS target
21    JOIN (
22        SELECT res.entry_id,
23        RANK() OVER (
24            ORDER BY TIMESTAMPDIFF(MICROSECOND, r.start_time, res.
25                end_time) ASC
26        ) AS pos
27    FROM RESULTS res
28    JOIN RACE_ENTRIES re ON res.entry_id = re.entry_id
29    JOIN RACES r ON re.race_code = r.race_code
30    WHERE re.race_code = p_race_code AND res.status = 'Finished'
31    ) rnk ON target.entry_id = rnk.entry_id
32    SET target.points = CASE
33        WHEN rnk.pos = 1 THEN 25 WHEN rnk.pos = 2 THEN 18
34        WHEN rnk.pos = 3 THEN 15 WHEN rnk.pos = 4 THEN 12
35        WHEN rnk.pos = 5 THEN 10 WHEN rnk.pos = 6 THEN 8
36        WHEN rnk.pos = 7 THEN 6 WHEN rnk.pos = 8 THEN 4
37        WHEN rnk.pos = 9 THEN 2 WHEN rnk.pos = 10 THEN 1
38        ELSE 0
39    END;
40
41    COMMIT;
42    END //
43    DELIMITER ;

```

Listing 4.3: Stored Procedure sp_calculate_points — Tính điểm F1 tự động

Lưu ý kỹ thuật: SP dùng **Subquery** (Derived Table trong FROM của UPDATE) thay vì CTE (WITH ... AS (...)) vì MariaDB/MySQL có hạn chế tương thích khi dùng CTE bên trong câu UPDATE. Kết quả hoàn toàn tương đương.

4.6 Views — Bảng ảo tổng hợp BXH

```

1 CREATE VIEW v_race_performance AS
2 SELECT re.race_code, c.driver_code, c.team_code,
3        res.status, res.laps_completed, res.points,
4        TIMESTAMPDIFF(MICROSECOND, r.start_time, res.end_time) /
5            1000000
6            AS finish_time_seconds,
7        r.start_time AS race_start_time
8 FROM RESULTS res
9 JOIN RACE_ENTRIES re ON re.entry_id = re.entry_id
10 JOIN RACES r ON re.race_code = r.race_code
11 JOIN CONTRACTS c ON re.contract_id = c.contract_id;

```

Listing 4.4: View v_race_performance — Nguồn dữ liệu gốc cho BXH

```

1 CREATE VIEW v_driver_standings AS
2 SELECT d.driver_code, d.name, d.nationality, t.name AS team_name,
3        SUM(v.points) AS total_score,
4        SUM(CASE WHEN v.status = 'Finished'
5            THEN v.finish_time_seconds ELSE 0 END) AS
6            total_season_time
7 FROM DRIVERS d
8 JOIN CONTRACTS c ON d.driver_code = c.driver_code AND c.is_active = 1
9 JOIN TEAMS t ON c.team_code = t.team_code
10 JOIN v_race_performance v ON v.driver_code = d.driver_code
11                            AND v.team_code = t.team_code
12 GROUP BY d.driver_code, d.name, d.nationality, t.name
13 ORDER BY total_score DESC, total_season_time ASC;

```

Listing 4.5: View v_driver_standings — BXH tay đua toàn mùa giải

```

1 CREATE VIEW v_team_standings AS
2 SELECT t.team_code, t.name AS team_name, t.brand,

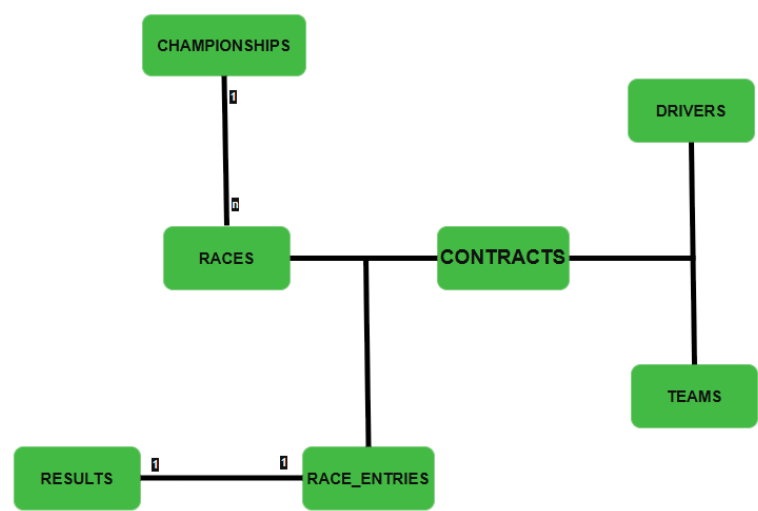
```

```
3      SUM(v.points) AS team_total_score ,
4      SUM(CASE WHEN v.status = 'Finished'
5            THEN v.finish_time_seconds ELSE 0 END) AS
              team_total_time
6 FROM TEAMS t
7 JOIN CONTRACTS c ON t.team_code = c.team_code AND c.is_active = 1
8 JOIN v_race_performance v ON v.team_code = t.team_code
9                        AND v.driver_code = c.driver_code
10 GROUP BY t.team_code, t.name, t.brand
11 ORDER BY team_total_score DESC, team_total_time ASC;
```

Listing 4.6: View v_team_standings — BXH đội đua toàn mùa giải

CHƯƠNG 5: THIẾT KẾ CƠ SỞ DỮ LIỆU

5.1 Mô hình thực thể — liên kết (ERD)



Hình 5.1: Sơ đồ ERD của hệ thống F1 Championship Management

5.2 Mô tả chi tiết các thực thể

Bảng 5.1: Bảng CHAMPIONSHIPS — Mùa giải

Cột	Kiểu	Ràng buộc	Mô tả
champ_code	VARCHAR(10)	PK	Mã mùa giải. VD: F1-2026
name	VARCHAR(255)	NOT NULL	Tên đầy đủ mùa giải
description	TEXT	NULL	Mô tả ngắn

Bảng 5.2: Bảng CONTRACTS — Hợp đồng tay đua — đội

Cột	Kiểu	Ràng buộc	Mô tả
contract_id	INT	PK AUTO_INCREMENT	ID hợp đồng
driver_code	VARCHAR(10)	FK → DRIVERS	Tay đua
team_code	VARCHAR(10)	FK → TEAMS	Đội đua
is_active	TINYINT	DEFAULT 1	1=hiệu lực, 0=hết hạn

Bảng 5.3: Bảng RESULTS — Kết quả thi đấu

Cột	Kiểu	Ràng buộc	Mô tả
entry_id	INT	PK, FK → RACE_ENTRIES	Tham chiếu 1-1
end_time	DATETIME(3)	NULL	Thời điểm về đích
laps_completed	INT	CHECK ≥ 0	Số vòng hoàn thành
status	ENUM	DEFAULT 'Finished'	Trạng thái kết thúc
points	INT	DEFAULT 0, CHECK ≥ 0	Điểm F1 (do SP cập nhật)

5.3 Mô hình quan hệ

```

CHAMPIONSHIPS(champ_code, name, description)
TEAMS(team_code, name, brand, owner, description)
DRIVERS(driver_code, name, date_of_birth, nationality)
CONTRACTS(contract_id, driver_code*, team_code*, is_active)
RACES(race_code, name, num_laps, start_time, champ_code*)
RACE_ENTRIES(entry_id, race_code*, contract_id*)
RESULTS(entry_id*, end_time, laps_completed, status, points)

```

* Dấu * biểu thị khóa ngoại

5.4 Chuẩn hóa (Normalization)

5.4.1 1NF — First Normal Form

Yêu cầu: Mỗi cột chứa giá trị nguyên tử, không có nhóm lặp, có PK.

Đánh giá: Tất cả 7 bảng đạt 1NF. Ví dụ: danh sách tay đua của một đội không được lưu trong một cột text dạng "VER,NOR,PIA" mà được lưu thành nhiều dòng riêng trong bảng CONTRACTS.

5.4.2 2NF — Second Normal Form

Yêu cầu: Đạt 1NF + không có phụ thuộc hàm bộ phận.

Đánh giá: Tất cả bảng đạt 2NF. RACE_ENTRIES dùng surrogate key entry_id thay vì khóa ghép (race_code, contract_id), loại bỏ khả năng vi phạm 2NF nếu thêm thuộc tính mới.

5.4.3 3NF — Third Normal Form

Yêu cầu: Đạt 2NF + không có phụ thuộc hàm bắc cầu.

Ví dụ vi phạm đã được xử lý: Nếu để cột `team_name` trong bảng `RACE_ENTRIES`:

`entry_id` $\rightarrow$ `contract_id` $\rightarrow$ `team_code` $\rightarrow$ `team_name`

`team_name` phụ thuộc bắc cầu qua `contract_id` $\rightarrow$ `team_code`. Giải pháp: loại bỏ, JOIN qua `CONTRACTS` $\rightarrow$ `TEAMS` khi cần.

5.4.4 BCNF — Boyce-Codd Normal Form

Yêu cầu: Mọi phụ thuộc hàm $X \rightarrow Y$ đều có X là superkey.

Đánh giá: Không có vi phạm. Trong `CONTRACTS`: `contract_id` $\rightarrow$ (`driver_code`, `team_code`, `is_active`) — `contract_id` là PK (superkey). Trong `RESULTS`: `entry_id` $\rightarrow$ (`end_time`, `status`, `points`) — `entry_id` là PK.

Kết luận: Schema đạt chuẩn BCNF, cân bằng tốt giữa tính chuẩn hóa và hiệu năng thực tế.

CHƯƠNG 6: TRIỂN KHAI

6.1 Môi trường và công nghệ

Bảng 6.1: Bảng công nghệ sử dụng

Thành phần	Công nghệ	Phiên bản
DBMS	MySQL / MariaDB	8.0 / 10.6+
Backend	Node.js + Express	18+
DB Driver	mysql2/promise	3.x
Frontend	React.js	18+
Quản trị DB	MySQL Workbench	8.x

6.2 Schema — Tạo cấu trúc bảng (DDL)

```
1 DROP DATABASE IF EXISTS F1_Championship_Management;
2 CREATE DATABASE F1_Championship_Management;
3 USE F1_Championship_Management;
4
5 SET SQL_SAFE_UPDATES = 0;
6
7 CREATE TABLE CHAMPIONSHIPS (
8     champ_code VARCHAR(10) PRIMARY KEY,
9     name        VARCHAR(255) NOT NULL,
10    description TEXT
11 );
12
13 CREATE TABLE TEAMS (
14     team_code  VARCHAR(10) PRIMARY KEY,
15     name       VARCHAR(100) NOT NULL,
16     brand      VARCHAR(100),
17     owner      VARCHAR(100),
18     description TEXT
19 );
20
21 CREATE TABLE DRIVERS (
22     driver_code VARCHAR(10) PRIMARY KEY,
23     name        VARCHAR(100) NOT NULL,
24     date_of_birth DATE,
```

```
25     nationality    VARCHAR(50),
26     biography      TEXT
27 );
28
29 CREATE TABLE CONTRACTS (
30     contract_id INT AUTO_INCREMENT PRIMARY KEY,
31     driver_code VARCHAR(10),
32     team_code VARCHAR(10),
33     is_active TINYINT DEFAULT 1,
34     FOREIGN KEY (driver_code) REFERENCES DRIVERS(driver_code),
35     FOREIGN KEY (team_code) REFERENCES TEAMS(team_code)
36 );
37
38 CREATE TABLE RACES (
39     race_code VARCHAR(10) PRIMARY KEY,
40     name VARCHAR(100) NOT NULL,
41     num_laps INT NOT NULL,
42     location VARCHAR(255),
43     start_time DATETIME(3),
44     champ_code VARCHAR(10),
45     description TEXT,
46     FOREIGN KEY (champ_code) REFERENCES CHAMPIONSHIPS(champ_code)
47 );
48
49 CREATE TABLE RACE_ENTRIES (
50     entry_id INT AUTO_INCREMENT PRIMARY KEY,
51     race_code VARCHAR(10),
52     contract_id INT,
53     FOREIGN KEY (race_code) REFERENCES RACES(race_code),
54     FOREIGN KEY (contract_id) REFERENCES CONTRACTS(contract_id),
55     UNIQUE(race_code, contract_id)
56 );
57
58 CREATE TABLE RESULTS (
59     entry_id INT PRIMARY KEY,
60     end_time DATETIME(3),
61     laps_completed INT,
62     status ENUM('Finished', 'DNF', 'Accident') DEFAULT '
        Finished',
63     points INT DEFAULT 0,
64     FOREIGN KEY (entry_id) REFERENCES RACE_ENTRIES(entry_id),
```

```

65     CONSTRAINT chk_laps_completed CHECK (laps_completed >= 0),
66     CONSTRAINT chk_points          CHECK (points >= 0)
67 );

```

Listing 6.1: DDL — Tạo toàn bộ cấu trúc bảng

6.3 Nạp dữ liệu mẫu (DML — INSERT)

```

1  INSERT INTO CHAMPIONSHIPS VALUES
2      ('F1-2025', 'Formula 1 Season 2025', 'M a g i i F1 2025'),
3      ('F1-2026', 'Formula 1 Season 2026', 'M a g i i F1 2026');
4
5  -- Doi dua (dung chung cho ca 2 mua)
6  INSERT INTO TEAMS VALUES
7      ('RBR', 'Red Bull Racing', 'Red Bull', 'Christian Horner', 'Doi
8          dua xuất sắc nhất thập kỷ'),
9      ('FER', 'Scuderia Ferrari', 'Ferrari', 'Fred Vasseur', 'Doi
10         dua lâu đời nhất F1'),
11      ('MER', 'Mercedes-AMG', 'Mercedes', 'Toto Wolff', '
12         Silver Arrows huyền thoại'),
13      ('MCL', 'McLaren', 'McLaren', 'Zak Brown', '
14         Papaya Power'),
15      ('ALP', 'Alpine F1 Team', 'Renault', 'Oliver Oakes', 'Doi
16         dua của Pháp'),
17      ('AST', 'Aston Martin', 'Mercedes', 'Lawrence Stroll', 'Doi
18         dua xanh lá');
19
20 -- Tay ua (18 tay dua - 3 tay dua moi doi)
21 INSERT INTO DRIVERS VALUES
22      ('VER', 'Max Verstappen', '1997-09-30', 'Dutch', '4x
23         World Champion, số 1 thập kỷ'),
24      ('NOR', 'Lando Norris', '1999-11-13', 'British', '
25         McLaren số 1 - đang ở đỉnh phong độ'),
26      ('PIA', 'Oscar Piastri', '2001-04-06', 'Australian', 'Tai
27         nang trẻ xuất sắc của McLaren'),
28      ('LEC', 'Charles Leclerc', '1997-10-16', 'Monegasque', '
29         Bieu tuong Ferrari - que hương Monaco'),
30      ('SAI', 'Carlos Sainz', '1994-09-01', 'Spanish', 'Il
31         Matador - chắc chắn, nhất quán'),
32      ('HAM', 'Lewis Hamilton', '1985-01-07', 'British', '7x
33         World Champion - huyền thoại F1'),

```

```

22      ('RUS', 'George Russell', '1998-02-15', 'British', 'Mr.
        Saturday - xuất sắc trong qualifying'),
23      ('ALO', 'Fernando Alonso', '1981-07-29', 'Spanish', '2x
        World Champion - vận thi đấu ở tuổi 44'),
24      ('STR', 'Lance Stroll', '1998-10-29', 'Canadian', 'Con
        trai ông chủ Aston Martin'),
25      ('GAS', 'Pierre Gasly', '1996-02-07', 'French', 'Cầu
        tay đua Red Bull - trụ cột Alpine'),
26      ('PER', 'Sergio Perez', '1990-01-26', 'Mexican', '
        Checo - Wing man hoàn hảo của Verstappen'),
27      ('BEA', 'Oliver Bearman', '2005-05-08', 'British', 'Tai
        nang trẻ người Anh - tương lai F1'),
28      ('LAW', 'Liam Lawson', '2002-02-11', 'New Zealander', '
        Người thay thế Perez tại Red Bull 2026'),
29      ('OWA', 'Pato O'Ward', '1999-05-06', 'Mexican', '
        Ngôi sao IndyCar chuyển sang F1 cùng Alpine'),
30      ('ANT', 'Kimi Antonelli', '2006-08-25', 'Italian', '
        Người kế thừa Hamilton tại Mercedes 2025'),
31      ('ZHO', 'Zhou Guanyu', '1999-05-30', 'Chinese', 'Tay
        đua người Trung Quốc - reserve McLaren'),
32      ('DEV', 'Jack Doohan', '2003-01-05', 'Australian', 'Hoc
        vien Alpine - reserve Alpine 2026'),
33      ('HUL', 'Nico Hulkenberg', '1987-08-19', 'German', 'Tay
        đua kỳ cựu - reserve Aston Martin');
34
35
36  -- HOP DONG 2025 (is_active = 0 vì đã kết thúc)
37  INSERT INTO CONTRACTS (driver_code, team_code, is_active) VALUES
38      ('VER', 'RBR', 0), ('PER', 'RBR', 0),
39      ('LEC', 'FER', 0), ('SAI', 'FER', 0),
40      ('HAM', 'MER', 0), ('RUS', 'MER', 0),
41      ('NOR', 'MCL', 0), ('PIA', 'MCL', 0),
42      ('GAS', 'ALP', 0),
43      ('ALO', 'AST', 0), ('STR', 'AST', 0);
44
45  -- HOP DONG MUA 2026 (is_active = 1 đang hiệu lực)
46  INSERT INTO CONTRACTS (driver_code, team_code, is_active) VALUES
47      ('VER', 'RBR', 1), ('LAW', 'RBR', 1), ('PER', 'RBR', 1),
48      ('HAM', 'FER', 1), ('LEC', 'FER', 1), ('SAI', 'FER', 1),
49      ('RUS', 'MER', 1), ('BEA', 'MER', 1), ('ANT', 'MER', 1),
50      ('NOR', 'MCL', 1), ('PIA', 'MCL', 1), ('ZHO', 'MCL', 1),

```

```

51      ('GAS', 'ALP', 1), ('OWA', 'ALP', 1), ('DEV', 'ALP', 1),
52      ('ALO', 'AST', 1), ('STR', 'AST', 1), ('HUL', 'AST', 1);
53
54  -- CHANG DUA MUA 2025
55  INSERT INTO RACES VALUES
56      ('BHR25', 'Bahrain Grand Prix', 57, 'Sakhir, Bahrain',
57          '2025-03-02 15:00:00.000', 'F1-2025', 'Season Opener 2025')
58      ,
59      ('JED25', 'Saudi Arabian Grand Prix', 50, 'Jeddah, Saudi Arabia',
60          '2025-03-16 17:00:00.000', 'F1-2025', 'Night Race'),
61      ('AUS25', 'Australian Grand Prix', 58, 'Melbourne, Australia',
62          '2025-03-30 15:00:00.000', 'F1-2025', 'Albert Park Circuit'),
63      ('JAP25', 'Japanese Grand Prix', 53, 'Suzuka, Japan',
64          '2025-04-06 14:00:00.000', 'F1-2025', 'Suzuka Classic'),
65      ('CHN25', 'Chinese Grand Prix', 56, 'Shanghai, China',
66          '2025-04-20 08:00:00.000', 'F1-2025', 'Shanghai
67          International'),
68      ('MIA25', 'Miami Grand Prix', 57, 'Miami, USA',
69          '2025-05-04 20:00:00.000', 'F1-2025', 'Hard Rock
70          Stadium'),
71      ('MONACO25', 'Monaco Grand Prix', 78, 'Monte Carlo, Monaco',
72          '2025-05-25 15:00:00.000', 'F1-2025', 'Jewel of the Crown'),
73      ('SPA25', 'Spanish Grand Prix', 66, 'Barcelona, Spain',
74          '2025-06-01 15:00:00.000', 'F1-2025', 'Circuit de Catalunya'
75      );
76
77  -- CHANG DUA MUA 2026
78  INSERT INTO RACES VALUES
79      ('BHR26', 'Bahrain Grand Prix', 57, 'Sakhir, Bahrain',
80          '2026-03-05 15:00:00.000', 'F1-2026', 'Season Opener 2026')
81      ,
82      ('JED26', 'Saudi Arabian Grand Prix', 50, 'Jeddah, Saudi Arabia',
83          '2026-03-19 17:00:00.000', 'F1-2026', 'Night Race 2026'),
84      ('AUS26', 'Australian Grand Prix', 58, 'Melbourne, Australia',
85          '2026-04-06 15:00:00.000', 'F1-2026', 'Albert Park'),
86      ('MONACO26', 'Monaco Grand Prix', 78, 'Monte Carlo, Monaco',
87          '2026-05-24 15:00:00.000', 'F1-2026', 'Jewel of the Crown 2026
88          '),
89      ('SPA26', 'Spanish Grand Prix', 66, 'Barcelona, Spain',
90          '2026-06-07 15:00:00.000', 'F1-2026', 'Circuit de Catalunya'
91      );

```

```

72
73 INSERT INTO RACE_ENTRIES (race_code, contract_id) VALUES
74     ('BHR25',1), ('BHR25',2), ('BHR25',3), ('BHR25',4), ('BHR25',5), ('
       BHR25',6), ('BHR25',7), ('BHR25',8), ('BHR25',9), ('BHR25',10), ('
       BHR25',11),
75     ('JED25',1), ('JED25',2), ('JED25',3), ('JED25',4), ('JED25',5), ('
       JED25',6), ('JED25',7), ('JED25',8), ('JED25',9), ('JED25',10), ('
       JED25',11),
76     ('AUS25',1), ('AUS25',2), ('AUS25',3), ('AUS25',4), ('AUS25',5), ('
       AUS25',6), ('AUS25',7), ('AUS25',8), ('AUS25',9), ('AUS25',10), ('
       AUS25',11),
77     ('JAP25',1), ('JAP25',2), ('JAP25',3), ('JAP25',4), ('JAP25',5), ('
       JAP25',6), ('JAP25',7), ('JAP25',8), ('JAP25',9), ('JAP25',10), ('
       JAP25',11),
78     ('CHN25',1), ('CHN25',2), ('CHN25',3), ('CHN25',4), ('CHN25',5), ('
       CHN25',6), ('CHN25',7), ('CHN25',8), ('CHN25',9), ('CHN25',10), ('
       CHN25',11),
79     ('MIA25',1), ('MIA25',2), ('MIA25',3), ('MIA25',4), ('MIA25',5), ('
       MIA25',6), ('MIA25',7), ('MIA25',8), ('MIA25',9), ('MIA25',10), ('
       MIA25',11),
80     ('MONACO25',1), ('MONACO25',2), ('MONACO25',3), ('MONACO25',4), ('
       MONACO25',5), ('MONACO25',6), ('MONACO25',7), ('MONACO25',8), ('
       MONACO25',9), ('MONACO25',10), ('MONACO25',11),
81     ('SPA25',1), ('SPA25',2), ('SPA25',3), ('SPA25',4), ('SPA25',5), ('
       SPA25',6), ('SPA25',7), ('SPA25',8), ('SPA25',9), ('SPA25',10), ('
       SPA25',11);
82
83 -- BHR25: NOR P1, PIA P2, VER P3, LEC P4, HAM P5, RUS P6, SAI P7, ALO
      P8, STR P9 | GAS ACC, PER DNF
84 INSERT INTO RESULTS VALUES
85 ( 1, '2025-03-02 16:37:12.000', 57, 'Finished', 0),
86 ( 2, NULL, 12, 'DNF', 0),
87 ( 3, '2025-03-02 16:37:28.000', 57, 'Finished', 0),
88 ( 4, '2025-03-02 16:38:02.000', 57, 'Finished', 0),
89 ( 5, '2025-03-02 16:37:45.000', 57, 'Finished', 0),
90 ( 6, '2025-03-02 16:37:58.000', 57, 'Finished', 0),
91 ( 7, '2025-03-02 16:36:45.000', 57, 'Finished', 0),
92 ( 8, '2025-03-02 16:36:58.000', 57, 'Finished', 0),
93 ( 9, NULL, 44, 'Accident', 0),
94 (10, '2025-03-02 16:38:18.000', 57, 'Finished', 0),
95 (11, '2025-03-02 16:38:35.000', 57, 'Finished', 0);

```

```

96
97 -- JED25: VER P1, HAM P2, NOR P3, LEC P4, PIA P5, RUS P6, SAI P7, ALO
    P8, STR P9 / GAS ACC, PER DNF
98 INSERT INTO RESULTS VALUES
99 (12, '2025-03-16 18:30:22.000', 50, 'Finished', 0),
100 (13, NULL, 18, 'DNF', 0),
101 (14, '2025-03-16 18:31:12.000', 50, 'Finished', 0),
102 (15, '2025-03-16 18:32:02.000', 50, 'Finished', 0),
103 (16, '2025-03-16 18:30:38.000', 50, 'Finished', 0),
104 (17, '2025-03-16 18:31:45.000', 50, 'Finished', 0),
105 (18, '2025-03-16 18:30:55.000', 50, 'Finished', 0),
106 (19, '2025-03-16 18:31:28.000', 50, 'Finished', 0),
107 (20, NULL, 30, 'Accident', 0),
108 (21, '2025-03-16 18:32:18.000', 50, 'Finished', 0),
109 (22, '2025-03-16 18:32:35.000', 50, 'Finished', 0);
110
111 -- AUS25: PIA P1, NOR P2, VER P3, LEC P4, HAM P5, SAI P6, RUS P7, ALO
    P8, GAS P9 / STR DNF, PER ACC
112 INSERT INTO RESULTS VALUES
113 (23, '2025-03-30 16:27:48.000', 58, 'Finished', 0),
114 (24, NULL, 12, 'Accident', 0),
115 (25, '2025-03-30 16:28:05.000', 58, 'Finished', 0),
116 (26, '2025-03-30 16:28:38.000', 58, 'Finished', 0),
117 (27, '2025-03-30 16:28:22.000', 58, 'Finished', 0),
118 (28, '2025-03-30 16:28:55.000', 58, 'Finished', 0),
119 (29, '2025-03-30 16:27:32.000', 58, 'Finished', 0),
120 (30, '2025-03-30 16:27:15.000', 58, 'Finished', 0),
121 (31, '2025-03-30 16:29:28.000', 58, 'Finished', 0),
122 (32, '2025-03-30 16:29:12.000', 58, 'Finished', 0),
123 (33, NULL, 25, 'DNF', 0);
124
125 -- JAP25: VER P1, NOR P2, PIA P3, HAM P4, LEC P5, RUS P6, SAI P7, ALO
    P8, GAS P9 / STR DNF, PER ACC
126 INSERT INTO RESULTS VALUES
127 (34, '2025-04-06 15:22:33.000', 53, 'Finished', 0),
128 (35, NULL, 0, 'Accident', 0),
129 (36, '2025-04-06 15:23:40.000', 53, 'Finished', 0),
130 (37, '2025-04-06 15:24:13.000', 53, 'Finished', 0),
131 (38, '2025-04-06 15:23:23.000', 53, 'Finished', 0),
132 (39, '2025-04-06 15:23:57.000', 53, 'Finished', 0),
133 (40, '2025-04-06 15:22:50.000', 53, 'Finished', 0),

```

```

134 (41, '2025-04-06 15:23:07.000', 53, 'Finished', 0),
135 (42, '2025-04-06 15:24:47.000', 53, 'Finished', 0),
136 (43, '2025-04-06 15:24:30.000', 53, 'Finished', 0),
137 (44, NULL, 28, 'DNF', 0);
138
139 -- CHN25: NOR P1, PIA P2, VER P3, LEC P4, SAI P5, HAM P6, RUS P7, ALO
    P8, STR P9 | GAS DNF, PER ACC
140 INSERT INTO RESULTS VALUES
141 (45, '2025-04-20 09:33:14.000', 56, 'Finished', 0),
142 (46, NULL, 0, 'Accident', 0),
143 (47, '2025-04-20 09:33:31.000', 56, 'Finished', 0),
144 (48, '2025-04-20 09:33:48.000', 56, 'Finished', 0),
145 (49, '2025-04-20 09:34:04.000', 56, 'Finished', 0),
146 (50, '2025-04-20 09:34:21.000', 56, 'Finished', 0),
147 (51, '2025-04-20 09:32:41.000', 56, 'Finished', 0),
148 (52, '2025-04-20 09:32:58.000', 56, 'Finished', 0),
149 (53, NULL, 11, 'DNF', 0),
150 (54, '2025-04-20 09:34:37.000', 56, 'Finished', 0),
151 (55, '2025-04-20 09:34:54.000', 56, 'Finished', 0);
152
153 -- MIA25: NOR P1, PIA P2, HAM P3, LEC P4, VER P5, RUS P6, SAI P7, ALO
    P8, GAS P9, STR P10 | PER DNF
154 INSERT INTO RESULTS VALUES
155 (56, '2025-05-04 21:33:07.000', 57, 'Finished', 0),
156 (57, NULL, 22, 'DNF', 0),
157 (58, '2025-05-04 21:32:50.000', 57, 'Finished', 0),
158 (59, '2025-05-04 21:33:40.000', 57, 'Finished', 0),
159 (60, '2025-05-04 21:32:33.000', 57, 'Finished', 0),
160 (61, '2025-05-04 21:33:23.000', 57, 'Finished', 0),
161 (62, '2025-05-04 21:32:00.000', 57, 'Finished', 0),
162 (63, '2025-05-04 21:32:17.000', 57, 'Finished', 0),
163 (64, '2025-05-04 21:34:30.000', 57, 'Finished', 0),
164 (65, '2025-05-04 21:33:57.000', 57, 'Finished', 0),
165 (66, '2025-05-04 21:34:13.000', 57, 'Finished', 0);
166
167 -- MONACO25: LEC P1, NOR P2, VER P3, PIA P4, HAM P5, RUS P6, SAI P7,
    ALO P8, GAS P9 | STR DNF, PER DNF
168 INSERT INTO RESULTS VALUES
169 (67, '2025-05-25 16:47:03.000', 78, 'Finished', 0),
170 (68, NULL, 22, 'DNF', 0),
171 (69, '2025-05-25 16:46:30.000', 78, 'Finished', 0), -

```

```

172 (70, '2025-05-25 16:48:10.000', 78, 'Finished', 0),
173 (71, '2025-05-25 16:47:37.000', 78, 'Finished', 0),
174 (72, '2025-05-25 16:47:53.000', 78, 'Finished', 0),
175 (73, '2025-05-25 16:46:47.000', 78, 'Finished', 0),
176 (74, '2025-05-25 16:47:20.000', 78, 'Finished', 0),
177 (75, '2025-05-25 16:48:43.000', 78, 'Finished', 0),
178 (76, '2025-05-25 16:48:27.000', 78, 'Finished', 0),
179 (77, NULL, 35, 'DNF', 0);
180
181 -- SPA25: NOR P1, VER P2, PIA P3, LEC P4, HAM P5, RUS P6, SAI P7, ALO
    P8, GAS P9 / STR DNF, PER ACC
182 INSERT INTO RESULTS VALUES
183 (78, '2025-06-01 16:34:50.000', 66, 'Finished', 0),
184 (79, NULL, 8, 'Accident', 0),
185 (80, '2025-06-01 16:35:23.000', 66, 'Finished', 0),
186 (81, '2025-06-01 16:36:13.000', 66, 'Finished', 0),
187 (82, '2025-06-01 16:35:40.000', 66, 'Finished', 0),
188 (83, '2025-06-01 16:35:57.000', 66, 'Finished', 0),
189 (84, '2025-06-01 16:34:33.000', 66, 'Finished', 0),
190 (85, '2025-06-01 16:35:07.000', 66, 'Finished', 0),
191 (86, '2025-06-01 16:36:47.000', 66, 'Finished', 0),
192 (87, '2025-06-01 16:36:30.000', 66, 'Finished', 0),
193 (88, NULL, 40, 'DNF', 0);
194
195 -- Tính điểm toán bo mùa 2025
196 CALL sp_calculate_points('BHR25');
197 CALL sp_calculate_points('JED25');
198 CALL sp_calculate_points('AUS25');
199 CALL sp_calculate_points('JAP25');
200 CALL sp_calculate_points('CHN25');
201 CALL sp_calculate_points('MIA25');
202 CALL sp_calculate_points('MONACO25');
203 CALL sp_calculate_points('SPA25');

```

Listing 6.2: INSERT — Dữ liệu mẫu mùa giải F1 2025 và F1 2026

6.4 Các câu truy vấn SQL quan trọng

```

1 SELECT re.entry_id, d.name AS driver_name,
2        t.name AS team_name, d.nationality
3 FROM RACE_ENTRIES re

```

```

4 JOIN CONTRACTS c ON re.contract_id = c.contract_id
5 JOIN DRIVERS d ON c.driver_code = d.driver_code
6 JOIN TEAMS t ON c.team_code = t.team_code
7 WHERE re.race_code = 'MONACO26'
8 ORDER BY t.name ASC, d.name ASC;

```

Listing 6.3: SELECT với JOIN — Lấy danh sách đăng ký một chặng

```

1 SELECT t.team_code, t.name AS team_name,
2        SUM(vp.points) AS team_score
3 FROM TEAMS t
4 JOIN CONTRACTS c ON t.team_code = c.team_code AND c.is_active = 1
5 JOIN v_race_performance vp ON vp.team_code = t.team_code
6 GROUP BY t.team_code, t.name
7 HAVING SUM(vp.points) > 50
8 ORDER BY team_score DESC;

```

Listing 6.4: SELECT với GROUP BY + HAVING — Đội có điểm trên 50

```

1 INSERT INTO RESULTS (entry_id, end_time, laps_completed, status)
2 VALUES (5, '2026-05-24 17:26:15.483', 78, 'Finished')
3 ON DUPLICATE KEY UPDATE
4     end_time          = VALUES(end_time),
5     laps_completed    = VALUES(laps_completed),
6     status            = VALUES(status);
7 -- points duoc cap nhap tu dong boi sp_calculate_points

```

Listing 6.5: INSERT với ON DUPLICATE KEY UPDATE — Upsert kết quả

```

1 SELECT d.driver_code, d.name, t.name AS team_name,
2        SUM(vp.points) AS total_score,
3        SUM(CASE WHEN vp.status = 'Finished'
4              THEN vp.finish_time_seconds ELSE 0 END) AS total_time
5 FROM DRIVERS d
6 JOIN CONTRACTS c ON d.driver_code = c.driver_code AND c.is_active = 1
7 JOIN TEAMS t ON c.team_code = t.team_code
8 JOIN v_race_performance vp ON vp.driver_code = d.driver_code
9 JOIN RACES r ON vp.race_code = r.race_code
10 WHERE r.start_time <= (
11     SELECT start_time FROM RACES WHERE race_code = 'JED26'
12 )
13 GROUP BY d.driver_code, d.name, t.name
14 ORDER BY total_score DESC, total_time ASC;

```

Listing 6.6: SELECT với Subquery — BXH đến một chặng cụ thể

6.5 Index — Tối ưu hóa hiệu năng

```

1 CREATE INDEX idx_race_code          ON RACE_ENTRIES(race_code);
2 CREATE INDEX idx_team_code_contracts ON CONTRACTS(team_code,
   driver_code);
3 CREATE INDEX idx_results_status_time ON RESULTS(status, end_time);
4 CREATE INDEX idx_races_start_time   ON RACES(start_time);
5 CREATE INDEX idx_races_champ_code   ON RACES(champ_code);

```

Listing 6.7: Tạo 4 index bổ sung tối ưu hiệu năng

Bảng 6.2: Chiến lược Index của hệ thống

Index	Bảng.Cột	Phục vụ truy vấn
idx_race_code	RACE_ENTRIES(race_code)	WHERE race_code = ? — dùng nhiều nhất
idx_team_code_contracts	CONTRACTS(team_code, driver_code)	JOIN + Trigger đếm tay đua
idx_results_status_time	RESULTS(status, end_time)	RANK() ORDER BY trong SP
idx_races_start_time	RACES(start_time)	BXH lọc theo mốc thời gian

CHƯƠNG 7: ĐÁNH GIÁ

7.1 Đánh giá thiết kế cơ sở dữ liệu

7.1.1 Điểm mạnh

Chuẩn hóa đạt BCNF: Schema đạt chuẩn BCNF với thiết kế tách bạch rõ ràng. Không có dữ liệu dư thừa — thông tin đội chỉ trong TEAMS, thông tin tay đua chỉ trong DRIVERS.

Mô hình hóa đúng thực tế nghiệp vụ: CONTRACTS phản ánh đúng bản chất hợp đồng trong F1 (lịch sử chuyển đội, is_active). RACE_ENTRIES tách biệt với RESULTS phản ánh đúng hai sự kiện có thời điểm khác nhau.

Phân tầng ràng buộc rõ ràng:

- Tầng DB: NOT NULL, UNIQUE, FK, CHECK, TRIGGER — không bỏ qua được
- Tầng Backend: Validation server-side, Prepared Statements
- Tầng Frontend: UX gợi ý và validation trước khi submit

Sử dụng đầy đủ các đối tượng DBMS:

Bảng 7.1: Tổng hợp các đối tượng DBMS trong hệ thống

Đối tượng	Số lượng	Mục đích
Table	7	Lưu trữ dữ liệu căn bản
View	3	Đóng gói truy vấn BXH
Stored Procedure	1	Logic tính điểm F1
Trigger	3	Ràng buộc nghiệp vụ
Index	4	Tối ưu hiệu năng
Transaction	Nhiều điểm	Đảm bảo tính ACID

7.2 Hạn chế hiện tại

H1 — Không lưu ngày bắt đầu/kết thúc hợp đồng

CONTRACTS chỉ có is_active (0/1), không có start_date và end_date. Không thể biết tay đua chuyển đội vào ngày cụ thể nào. *Cải thiện:* ALTER TABLE CONTRACTS ADD COLUMN start_date DATE, ADD COLUMN end_date DATE;

H2 — Race Condition tại Trigger

Nếu 2 nhân viên đồng thời đăng ký tay đua thứ 2 của cùng đội, cả 2 transaction đọc count = 1 và đều pass trigger. Kết quả: đội có 3 tay đua. *Giải pháp:* Dùng SELECT ... FOR UPDATE để lock dòng trước khi kiểm tra.

H3 — Chưa có Authentication/Authorization

Hệ thống hiện không có đăng nhập và phân quyền. Mọi người đều có thể nhập kết quả.

H4 — Chưa có Audit Log

Không ghi lại lịch sử ai đã thay đổi dữ liệu. Cần thêm bảng AUDIT_LOG + Trigger ghi lại thay đổi.

7.3 Khả năng mở rộng

- **Ngắn hạn:** Thêm Authentication (JWT), giải quyết Race Condition, thêm Audit Log, thêm ngày bắt đầu/kết thúc hợp đồng.
- **Dài hạn:** Hỗ trợ nhiều mùa giải với bảng SCORING_SYSTEMS theo từng era F1. Export BXH PDF/Excel. Tích hợp MySQL Replication (Read Replica) để tách tải đọc và ghi.

CHƯƠNG 8: KẾT LUẬN

8.1 Tóm tắt kết quả

Báo cáo đã trình bày toàn diện quá trình thiết kế và triển khai hệ thống F1 Championship Management với MySQL/MariaDB. Kết quả đạt được:

- Schema 7 bảng đạt chuẩn BCNF, phản ánh đúng thực tế nghiệp vụ F1.
- 6 Foreign Key đảm bảo toàn vẹn tham chiếu hoàn chỉnh trong chuỗi dữ liệu.
- 3 Trigger thực thi ràng buộc nghiệp vụ tại tầng DB (giới hạn 2 tay đua/đội, thời gian hợp lệ).
- 1 Stored Procedure với Transaction ACID để tính điểm F1 tự động và nhất quán.
- 3 View đóng gói logic truy vấn phức tạp, API chỉ cần 1 câu SELECT.
- 4 Index bổ sung tối ưu hiệu năng cho các truy vấn quan trọng nhất.

8.2 Bài học rút ra

1. **Database là nguồn sự thật cuối cùng (Single Source of Truth):** Constraints và Trigger ở DB không thể bị bypass từ bất kỳ hướng nào — đây là lớp bảo vệ quan trọng nhất.
2. **Thiết kế schema chuẩn hóa tốt ngay từ đầu:** Tách RESULTS khỏi RACE_ENTRIES, dùng CONTRACTS thay vì lưu team_code trong DRIVERS giúp tránh hàng loạt vấn đề nhất quán dữ liệu về sau.
3. **Transaction cho mọi nghiệp vụ đa bước:** Bất kỳ nghiệp vụ nào liên quan đến nhiều bảng đều cần Transaction.
4. **Window Function RANK() là công cụ mạnh:** Giải quyết bài toán xếp hạng thanh lịch và hiệu quả $O(n \log n)$ so với subquery đếm $O(n^2)$.
5. **Index cần chọn lọc, không thừa:** Chỉ tạo index trên các cột xuất hiện trong WHERE, JOIN, ORDER BY của truy vấn quan trọng.
6. **Tính toán đúng tầng:** Đặt logic tính điểm vào Stored Procedure (tầng DB) thay vì code Node.js giúp DB tự bảo vệ tính nhất quán.

8.3 Hướng phát triển

- Thêm JWT Authentication và role-based access control
- Giải quyết Race Condition bằng `SELECT . . . FOR UPDATE`
- Thêm bảng lưu lịch sử thay đổi dữ liệu (Audit Log)
- Hỗ trợ nhiều hệ thống tính điểm theo từng era lịch sử F1
- Tích hợp Read Replica để scale khi traffic tăng cao

TÀI LIỆU THAM KHẢO

1. Ramakrishnan, R., & Gehrke, J. (2003). *Database Management Systems* (3rd ed.). McGraw-Hill.
2. Elmasri, R., & Navathe, S. B. (2015). *Fundamentals of Database Systems* (7th ed.). Pearson.
3. Oracle Corporation. (2024). *MySQL 8.0 Reference Manual*. <https://dev.mysql.com/doc/refman/8.0/en/>
4. MariaDB Foundation. (2024). *MariaDB Server Documentation*. <https://mariadb.com/kb/en/documentation/>
5. Codd, E. F. (1970). A relational model of data for large shared data banks. *Communications of the ACM*, 13(6), 377–387.
6. Fédération Internationale de l'Automobile (FIA). (2024). *Formula 1 Sporting Regulations*. <https://www.formula1.com/>